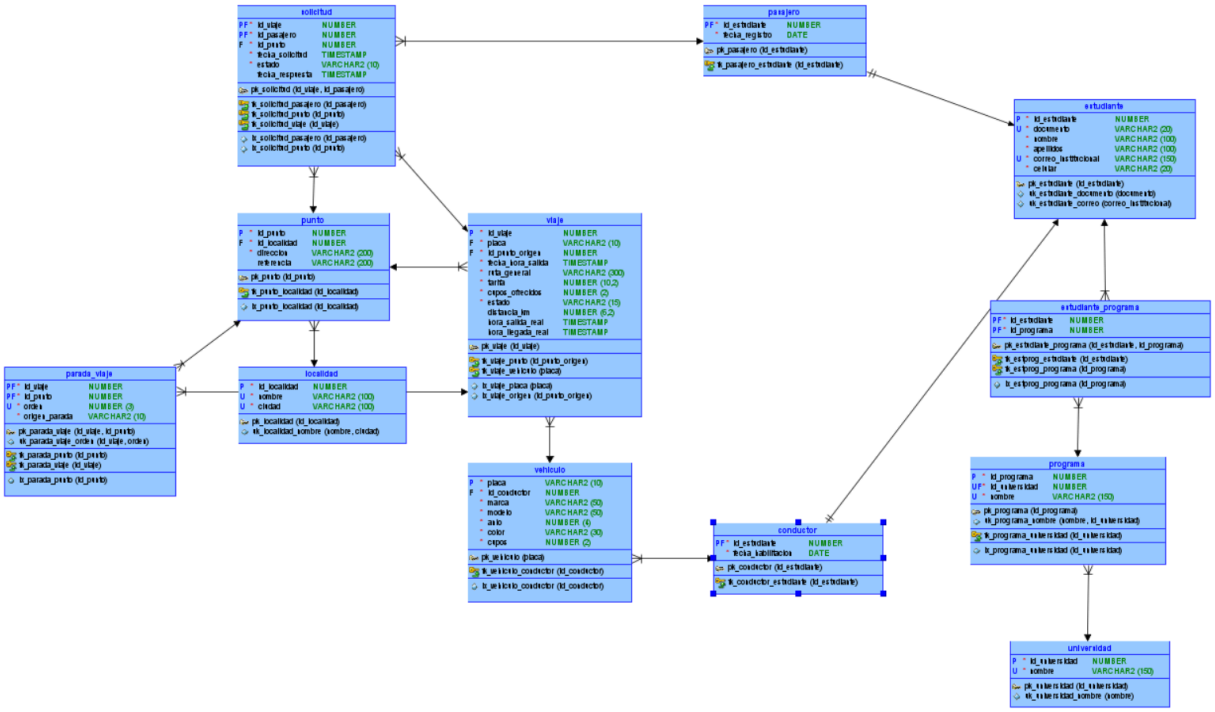
Actividad 1

1. Retroalimentación
   En la entrega 1, el monitor señaló para corregir cinco puntos. La redundancia en el modelo y la forma de modelar las relaciones. Relaciones del conductor omitidas por estar generalizadas sobre el estudiante. Un estudiante puede cursar varios programas. Un conductor debe tener al menos un vehículo. Y las restricciones del chequeo estaban identificadas, pero sin un dominio definido.

2. Correcciones realizadas

Comentario	Corrección
Redundancia	Se eliminó la relación entre punto y viaje porque el vínculo ya existe por viaje.id_punto_origen y por parada viaje. También se quitó el atributo sin tipo definido en parada_viaje y las columnas de llave foránea duplicadas en el diagrama y se unificó el nombre de la llave de punto como id_punto.
Relaciones sobre la generalización	Cada relación quedó asociada al subtipo que corresponde: vehículo con conductor y solicitud con pasajero. No se creó una relación directa entre conductor y viaje, ya que se llega a esta a través del vehículo y duplicarla genera la redundancia señalada.
Varios programas por estudiante	La relación programa-estudiante pasó de ser de 1:N a M:N, lo que crea la entidad estudiante_programa.
Conductor con al menos un vehículo	La relación conductor-vehículo queda de 1:N y obligatoriedad del lado del conductor.
Cambio de documento	Como se exigió por modificar el documento de identidad, el documento dejó de ser llave primaria del estudiante y se agregó la llave id_estudiante y documento como llave candidata.

3. Diagrama E/R corregido



4. Clases creadas y actualizadas
   - ❖ Creada: estudiante_programa.
   - ❖ Actualizadas: estudiante (nueva llave primaria, se elimina id_programa), conductor y pasajero (su llave pasa a ser id_estudiante), vehículo (id_conductor reemplaza a documento_conductor) y solicitud (su llave compuesta usa id_pasajero).
   - ❖ Sin cambios: universidad, programa, localidad, punto, viaje y parada viaje.

5. Modelo relacional
   El modelo relacional se actualizó aplicando las reglas de transformación vistas en clase y el documento está en la plantilla Excel. Esto quedó con las 12 tablas y los cambios clave son:
   - ❖ Pasajero y conductor conservan la llave de estudiante como PK y FK a la vez.
   - ❖ La relación M:N entre estudiante y programa se rompe con la tabla estudiante_programa, cuya PK es id_estudiante y id_programa.

6. Verificación de forma normal de Boyce-Codd

Tabla	Determinantes	Resultado
universidad	id_universidad, nombre	BCNF
programa	id_programa, (nombre, id_universidad)	BCNF
estudiante	id_estudiante, documento, correo_institucional	BCNF
estudiante_programa	(id_estudiante, id_programa)	BCNF
conductor, pasajero	id_estudiante	BCNF
vehículo	placa	BCNF
viaje, punto	llave primaria	BCNF
localidad	id_localidad, (nombre, ciudad)	BCNF
parada_viaje	(id_viaje, id_punto), (id_viaje, orden)	BCNF
solicitud	(id_viaje, id_pasajero)	BCNF

En todas las tablas cada determinante es una superclave, así que ninguna requirió una descomposición de tabla adicional. De esa manera se cumple el BCNF en las tablas.

7. Restricciones de dominio

Restricción	Expresión
viaje - estado	Estado (‘programado’, ‘en curso’, ‘completado’, ‘cancelado’)
solicitud - estado	Estado (‘pendiente’, ‘aceptada’, ‘rechazada’, ‘cancelada’)
vehículo - cupos	Entre 1 y 20
vehículo - año	Entre 1950 y 2030
viaje - tarifa	Tarifa >= 0
viaje - cupos	Cupos ofrecidos > 0

Diseño de sentencias SQL

1. Criterios usados
   Para implementar los requerimientos de consulta se tomaron las siguientes decisiones:
   - La localidad del punto de recogida que eligió el pasajero se obtiene con solicitud.id_ punto que referencia a punto y este a localidad.
   - La localidad de un viaje es el punto de origen.
   - En RFC 1 se filtra por fecha de solicitud. En RFC 2 y 3 se filtra por fecha de salida del viaje.
   - RFC 1 cuenta las solicitudes en cualquier estado porque mide la demanda. RFC 2 solo suma solicitudes aceptadas de viajes completados porque los ingresos solo existen si el viaje se realizó. RFC 3 solo cuenta viajes completados ya que pide los viajes realizados.
   - Se usa el límite inferior incluido y el superior excluido, como las fechas guardan hora, así se incluye el último día completo.
   - El enunciado menciona de RFC 1 a RFC 5, pero solo define los tres requerimientos, por lo que solo se implementó RFC de 1 a 3.

2. Descripción de las consultas

   RFC1

Que responde	Cuántas solicitudes de cupo hay por localidad en un rango de fechas
Tablas	solicitud, punto, localidad
Joins	solicitud.id_punto = punto.id_punto punto.id_localidad =localidad.id_localidad
Tipo de join	Inner Join
Filtro y agrupación	fecha_solicitud dentro del rango, agrupado por localidad

RFC2

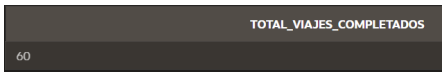
Que responde	Ingresos estimados por localidad en un rango de fechas
Tablas	solicitud, viaje, punto, localidad
Joins	solicitud.id_viaje = viaje.id_viaje solicitud.id_punto = punto.id_punto punto.id_localidad =localidad.id_localidad
Tipo de join	Inner Join
Filtro y agrupación	Solicitud aceptada, viaje completado, fecha de salida en el rango y SUM(tarifa) por localidad

RFC3

Que responde	Evolución semanal de los viajes realizados por cada conductor
Tablas	vehículo, viaje, conductor, punto, localidad, estudiante
Joins	viaje.placa = vehiculo.placa vehiculo.id_conductor = conductor.id_estudiante conductor.id_estudiante = estudiante.id_estudiante viaje.id_punto_origen = punto.id_punto punto.id_localidad =localidad.id_localidad
Tipo de join	Inner Join
Filtro y agrupación	Viaje completado en el rango y opcionalmente un conductor y una localidad Agrupado por conductor y semana

Justificación del tipo de join
En las tres consultas se usó Inner Join. Todas las llaves foráneas involucradas son obligatorias. Cada solicitud tiene un punto de recogida, cada punto tiene una localidad, cada viaje tiene un vehículo y cada vehículo tiene un conductor. Por eso el inner join no descarta las filas válidas. Además, los requerimientos solo muestran las localidades y los conductores con actividad en el periodo, de modo que un Left Join traería también los que tienen cero, lo cual no aporta información.

3. Verificación de los resultados
   Para comprobar que las consultas sean correctas, para cada una ejecutamos una segunda consulta que obtiene el total directamente de las tablas sin agrupar ni unir por localidad y se compararon ambos valores dentro de un rango específico de tiempo.

Requerimiento	Consulta	Verificación
RFC1 - solicitudes		
RFC2 - ingresos		
RFC3 - viajes		

Desarrollo de la aplicación

1. Arquitectura

La aplicación “Wheels Alpes” (ID 165442) se desarrolló en APEX sobre la base de datos del workspace SARA_WORKS. y tiene tres capas:

- **Base de datos:** las 12 tablas con sus restricciones de integridad (PK, FK, UNIQUE, CHECK) y los triggers que hacen cumplir las reglas del negocio (cupos, estados, cancelación con 2 horas).
- **Consultas:** los requerimientos de consulta RFC1 a RFC3 y los tableros RFA1 y RFA2, escritos en SQL y conectados a las páginas.
- **Páginas:** formularios, reportes y gráficas, con procesos PL/SQL que insertan o actualizan en la base de datos.

Autenticación y roles

Se usa una autenticación propia. El esquema “Login Estudiantes” (tipo Custom) llama a la función f_autenticar, que valida el correo institucional y la contraseña guardados en la tabla ESTUDIANTE. Para esto se agregó a ESTUDIANTE el atributo contrasena.

Al iniciar sesión, una computación de la aplicación guarda el id_estudiante en el ítem AI_ID_ESTUDIANTE, y todas las páginas lo usan para mostrar solo los datos de quien entró. Dos esquemas de autorización deciden el rol:

- **Es Conductor:** el estudiante existe en la tabla CONDUCTOR.
- **Es Pasajero:** el estudiante existe en la tabla PASAJERO.

Un mismo estudiante puede tener los dos roles, porque la herencia es con traslape, y en ese caso ve las opciones de ambos.

Navegación por rol

- **Todos, incluso sin iniciar sesión:** el login, los registros de conductor y de pasajero, y Buscar Viajes. Desde la pantalla de login hay enlaces a esas tres páginas.
- **Conductor:** Los Vehículos, Publicar Viaje, Mis Viajes, Gestionar Viaje, RFC1, RFC2, RFC3 y el Tablero Conductor.
- **Pasajero:** Mi Cuenta, Solicitar Cupo, Mis Solicitudes y el Tablero Pasajero.

El esquema de autorización está tanto en la entrada del menú como en la página misma. Si un usuario escribe a mano la URL de una página que no es de su rol, APEX no la abre y muestra “Esta opción es solo para conductores” (o pasajeros). La página Home muestra los accesos según el rol de quien entró.

Decisiones de diseño

- Las reglas del negocio están en los triggers y no solo en la interfaz, así que se cumplen aunque alguien inserte datos sin pasar por la app. Cuando se violan, la aplicación muestra el mensaje de error de Oracle del trigger, por ejemplo “RN4: el viaje solo se puede cancelar con al menos 2 horas de anticipación”.
- El registro de un conductor inserta en una sola transacción al estudiante, su programa, el conductor y su vehículo, porque RN10 exige que tenga al menos uno y eso no se puede garantizar con una restricción de la base. Si algo falla, no se guarda nada.
- Los formularios de registro se arman sobre vistas (v_conductor_vehiculo y v_estudiante_programa) que juntan las tablas necesarias. El guardado lo hace un proceso PL/SQL con los INSERT o UPDATE a cada tabla.
- El servidor de oracleapex.com trabaja en hora UTC, cinco horas adelante de Colombia. Por eso la regla de las 2 horas compara contra la hora de Bogotá: CAST(SYSTIMESTAMP AT TIME ZONE 'America/Bogota' AS TIMESTAMP).
- Los RFC y los tableros sólo los ve el usuario objetivo. Los RFC y el Tablero Conductor los ven los **conductores**, y el Tablero Pasajero los **pasajeros**. El enunciado no dice quién es el usuario objetivo de los RFC, así que se asignaron a los conductores.

2. Páginas de la aplicación

Página	**Requerimiento**	**Quién la usa**	**Descripción**
Registro Conductor	RF1	Nuevo usuario	Formulario de datos personales y del vehículo
Mis Vehículos	RF2	Conductor	Lista, alta y edición de sus vehículos
Publicar Viaje	RF3	Conductor	Formulario con vehículo, origen, fecha, ruta, tarifa, cupos y paradas
Gestionar Viaje	RF4, RF5	Conductor	Aceptar o rechazar solicitudes y cambiar el estado del viaje
Mis Viajes	RF6	Conductor	Historial con estado y pasajeros confirmados
Registro Pasajero	RF7	Nuevo usuario	Formulario de datos personales
Mi Cuenta	RF8	Pasajero	Modificar datos personales
Buscar Viajes	RF9	Público	Búsqueda por fecha y zona, sin sesión

Solicitar Cupo	RF10	Pasajero	Elegir viaje y punto de recogida
Mis Solicitudes	RF11	Pasajero	Consultar y cancelar solicitudes
RFC1, RFC2, RFC3	Consultas	Conductor	Reportes con gráficas
RFA1, RFA2	Tableros	Conductor, pasajero	Indicadores del usuario

3. Acción del requerimiento sobre la base de datos

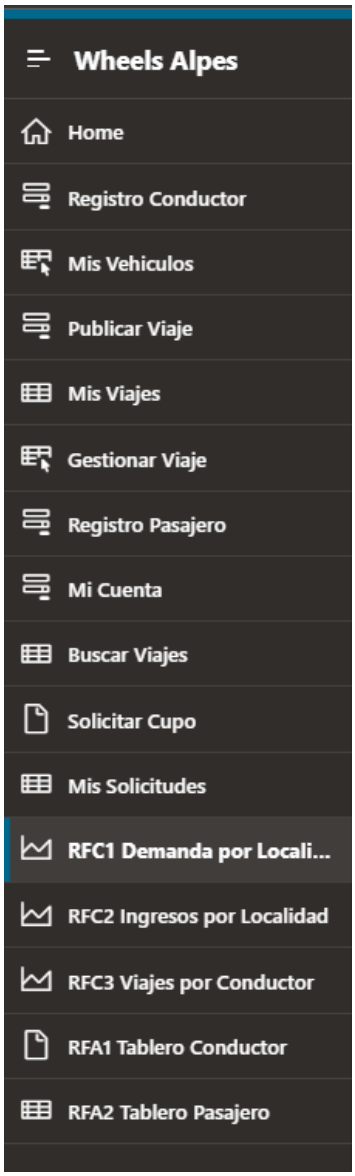
Operación	**Reglas que intervienen**
Inserta estudiante, programa del estudiante, conductor y vehículo en una transacción	Documento y correo únicos; al menos un vehículo
Inserta y actualiza en vehículo	Placa única; check de año y cupos
Inserta en viaje (estado PROGRAMADO) y sus paradas iniciales en parada_viaje	Cupos del viaje ≤ cupos del vehículo; al menos una parada
Actualiza el estado de la solicitud a ACEPTADA o RECHAZADA y guarda la fecha de respuesta	Cupos suficientes; al aceptar, el punto de recogida pasa a ser parada
Actualiza el estado del viaje y, al completarlo, la hora real de llegada y la distancia	Secuencia válida; cancelar con 2 horas; se cancelan sus solicitudes
Consulta viajes del conductor con pasajeros confirmados	Solo ve los suyos
Inserta estudiante, programa y pasajero	Documento y correo únicos
Actualiza estudiante y su programa	El documento se puede modificar porque la llave es id_estudiante; correo único
Consulta viajes PROGRAMADO por fecha y localidad	Sin autenticación

Inserta en solicitud con estado PENDIENTE	Viaje programado; el conductor no pide cupo en su viaje; una solicitud por viaje
Cambia la solicitud a CANCELADA	Con 2 horas de anticipación

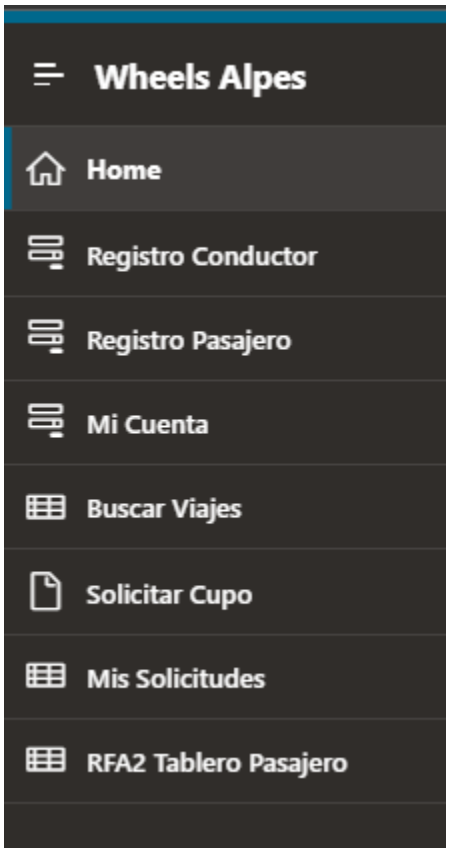
4. Consultas

- **RFC1, RFC2 y RFC3:** las consultas son las de la sección 4.
    - RFC1 y RFC2 se presentan como gráficas de barras por localidad y se filtran por fecha desde y hasta.
    - RFC3 se presenta como una gráfica de líneas, una línea por conductor y un punto por semana. Se filtra por conductor, localidad de origen y rango de fechas.
- **RFA1:** muestra pasajeros transportados, ingresos, distancia total y tiempo total de los viajes completados del conductor.
- **RFA2:** muestra viajes tomados, zona más frecuente, gasto total, y promedio de distancia y tiempo por viaje. Si hay empate en la zona más frecuente, se toma la primera en orden alfabético.

Menú de usuario con dos roles

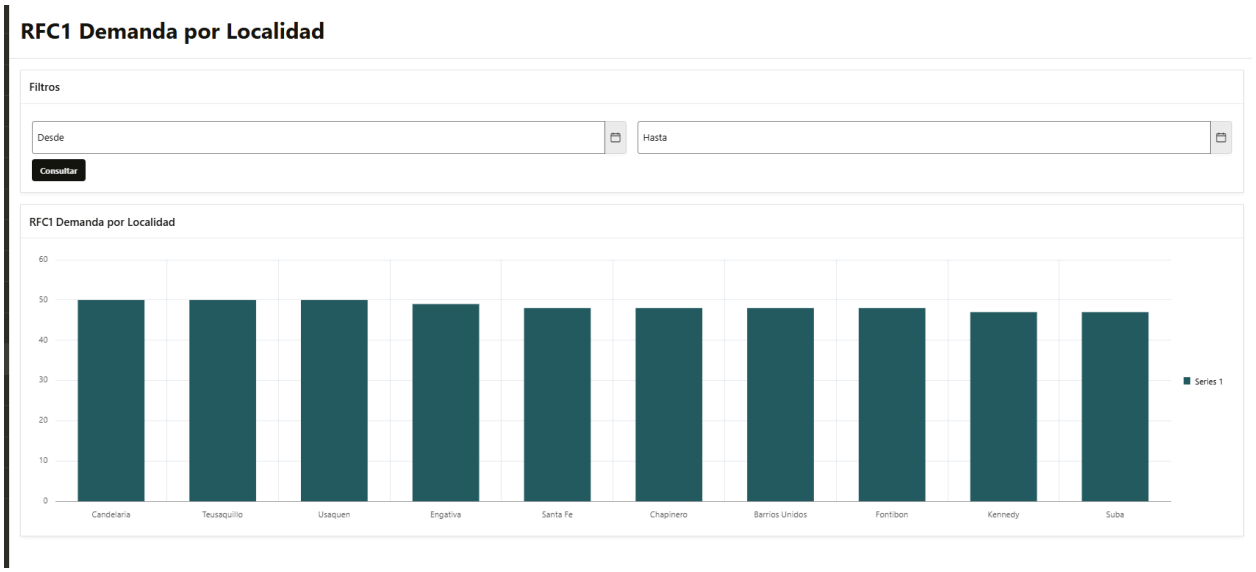


Menú pasajero

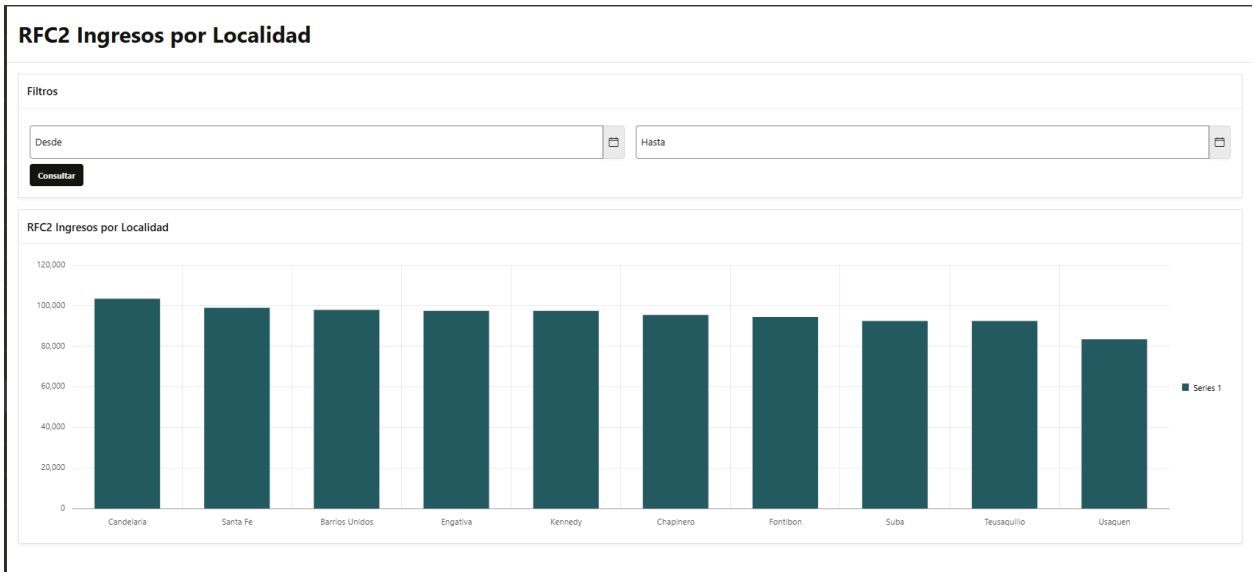


Capturas RFC

RFC1



RFC2



RFC3



Capturas RFA

RFA1

RFA1 Tablero Conductor

RFA1 Tablero Conductor

Viajes Completados	Pasajeros Transportados	Ingresos	Distancia Total Km	Tiempo Total Min
4	9	49500	64.5	115

RFA2

RFA2 Tablero Pasajero

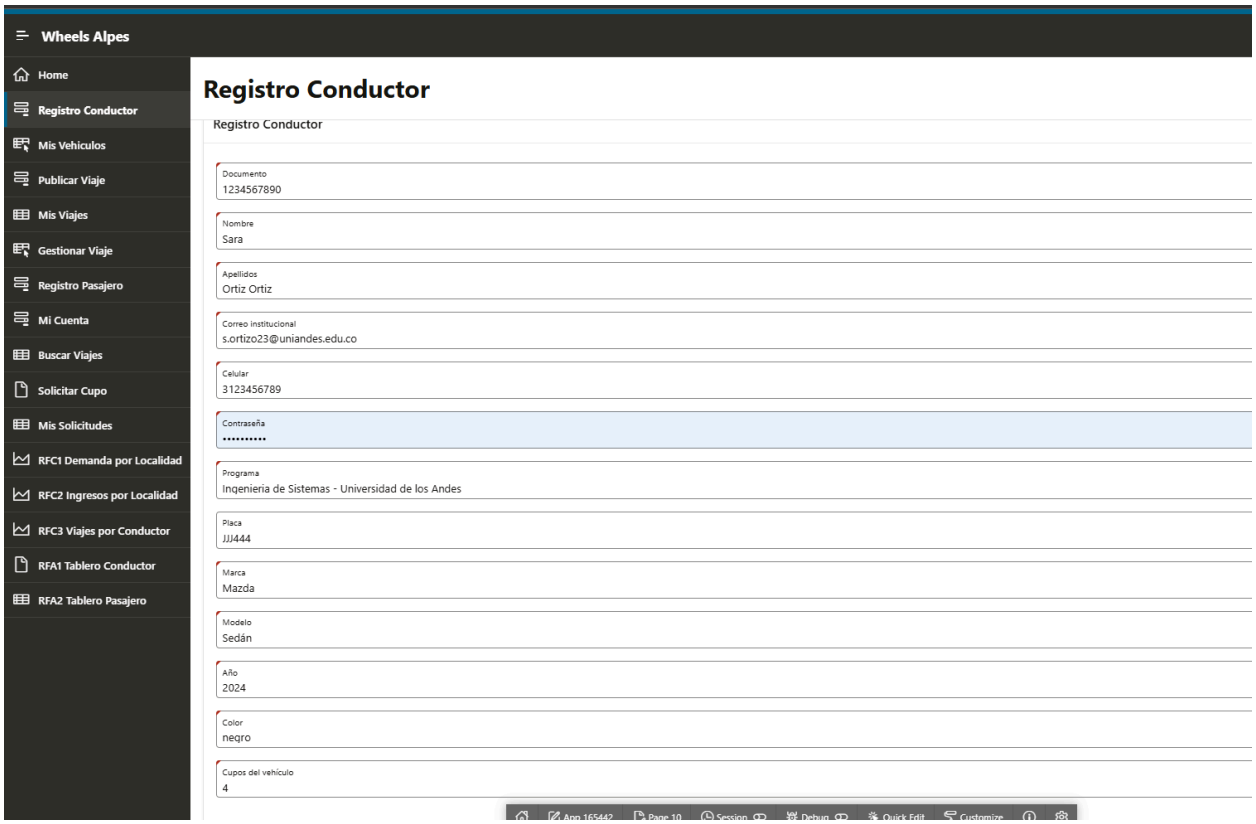
RFA2 Tablero Pasajero

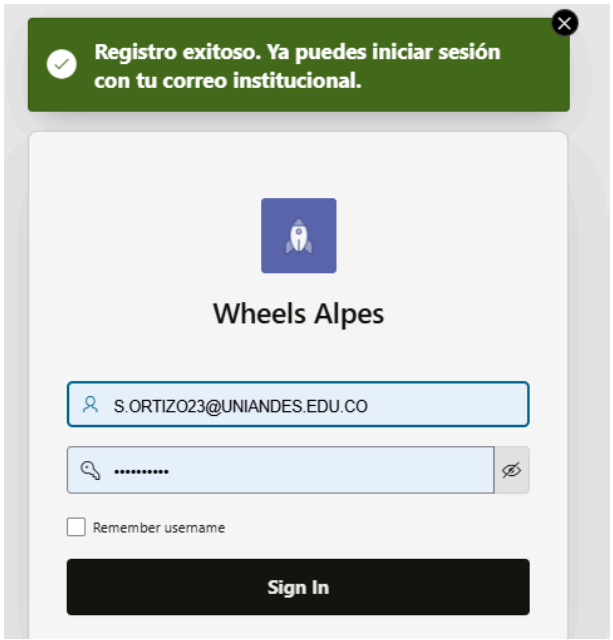
Viajes Tomados	Zona Mas Frecuente	Gasto Total	Distancia Promedio Km	Tiempo Promedio Min
0	Usaquen	0	0	0

Pruebas

Registro del conductor

Prueba exitosa





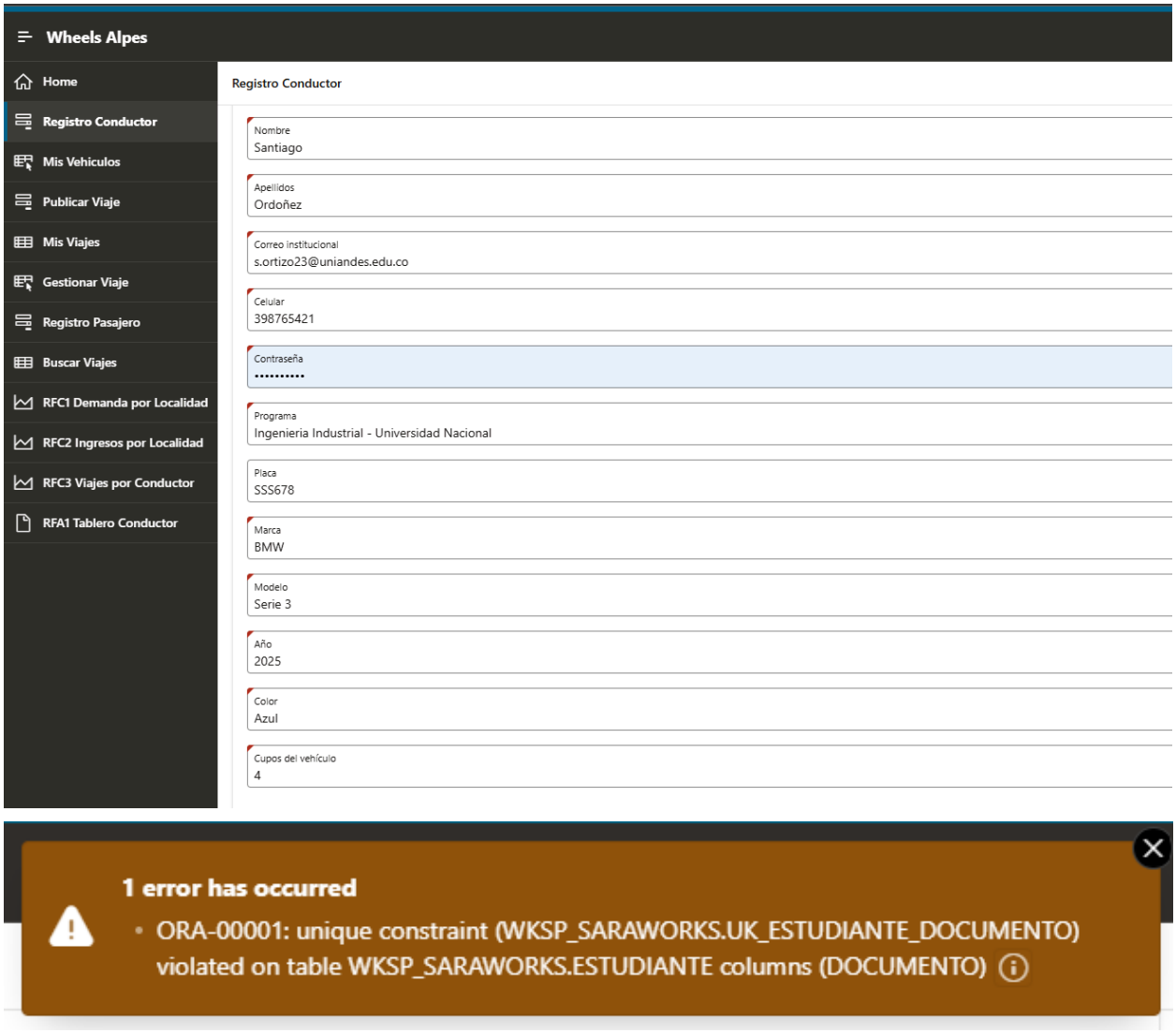
```sql
SELECT e.id_estudiante, e.documento, e.nombre, e.correo_institucional,
       c.fecha_habilitacion, v.placa, v.marca
  FROM estudiante e
  JOIN conductor c ON c.id_estudiante = e.id_estudiante
  JOIN vehiculo  v ON v.id_conductor  = c.id_estudiante
 WHERE e.documento = '1234567890';
```

Results | Explain | Describe | Saved SQL | History

ID_ESTUDIANTE	DOCUMENTO	NOMBRE	CORREO_INSTITUCIONAL	FECHA_HABILITACION	PLACA	MARCA
461	1234567890	Sara	s.ortizo23@uniandes.edu.co	11/3/2026	JJJ444	Mazda

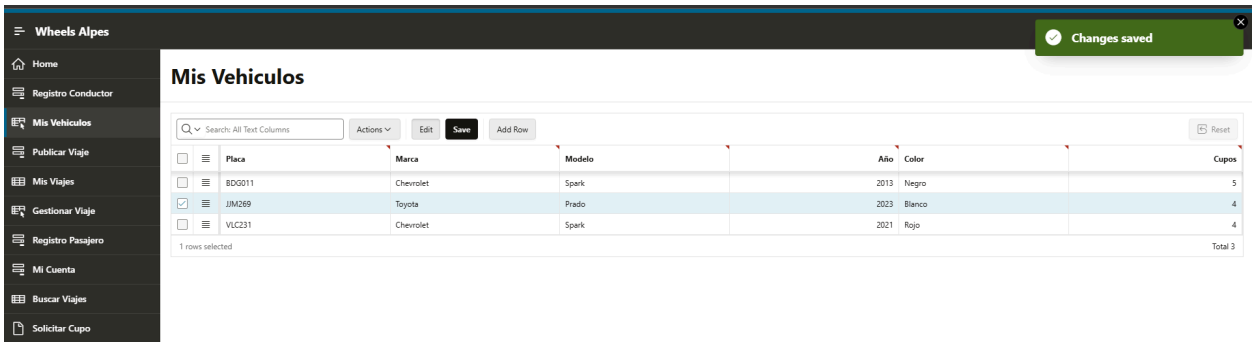
1 rows returned in 0.03 seconds Download

Prueba fallida

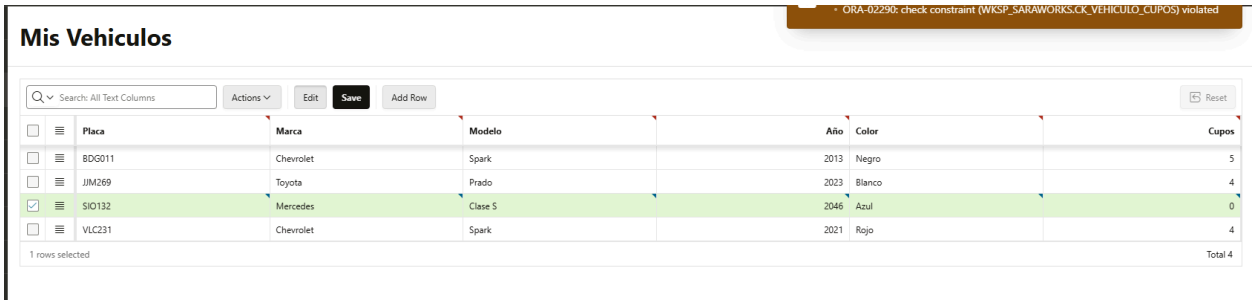


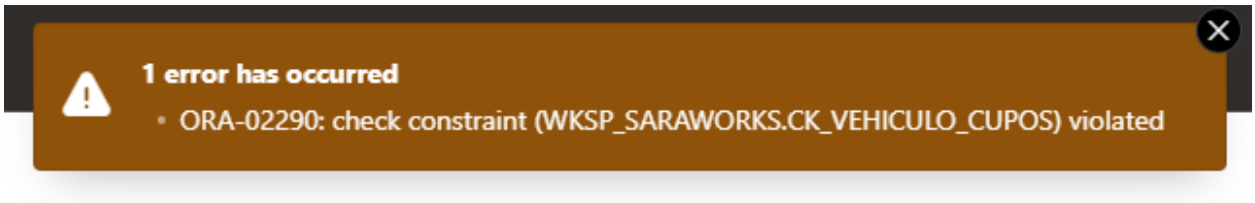
Ingresar un nuevo vehículo

Prueba exitosa



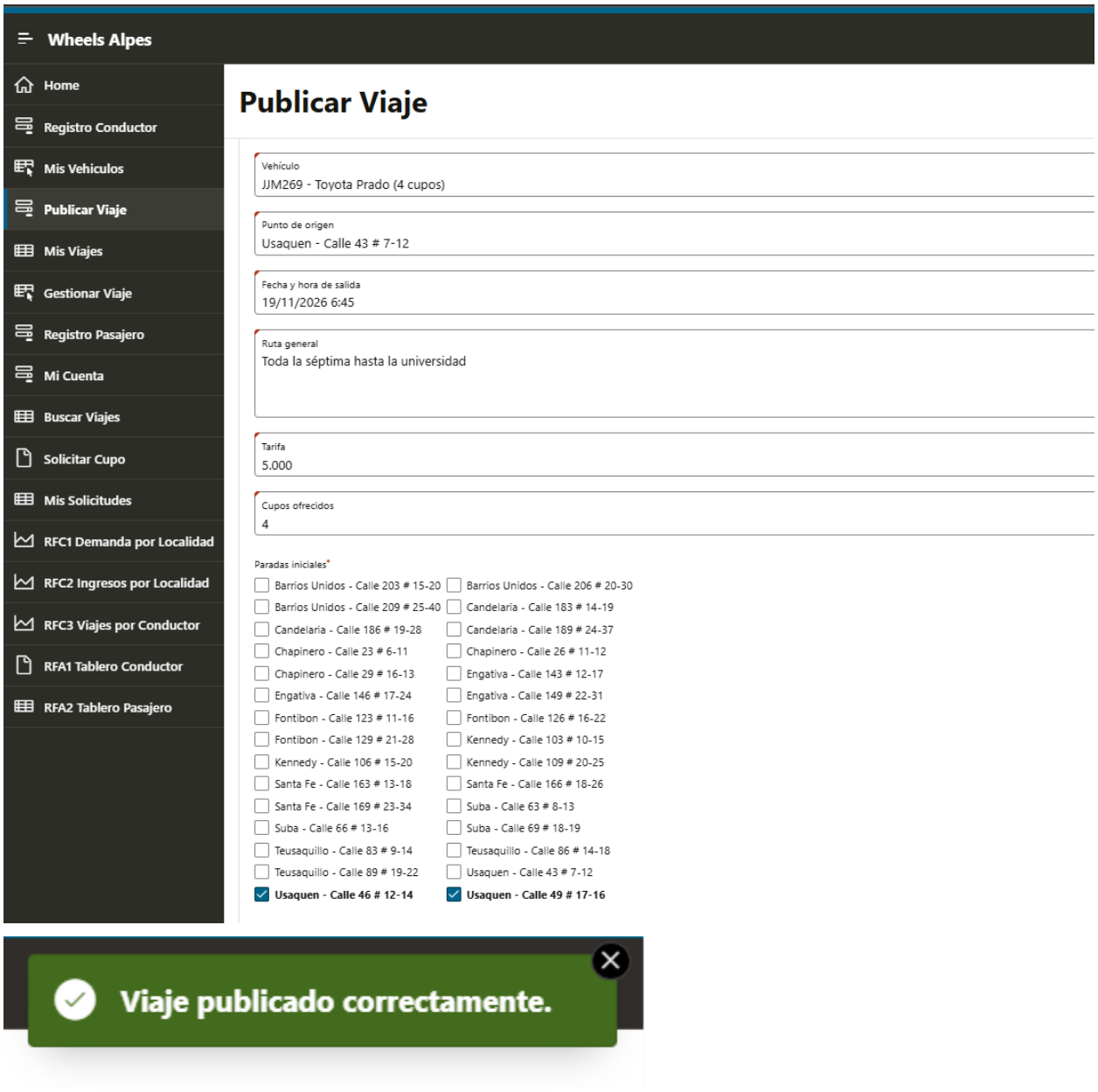
Prueba fallida



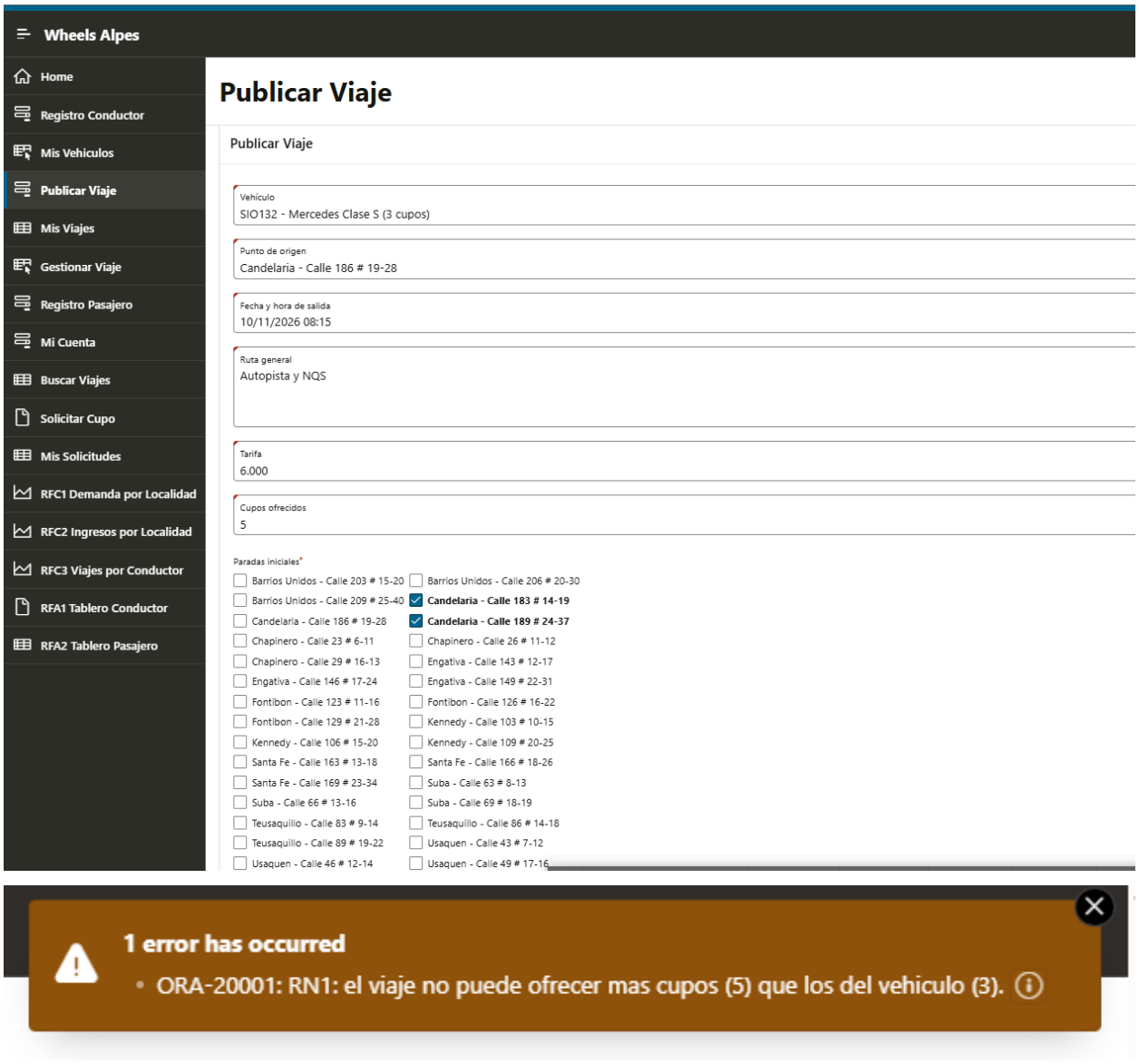


Publicar viaje

Prueba exitosa



Prueba fallida



Mis viajes

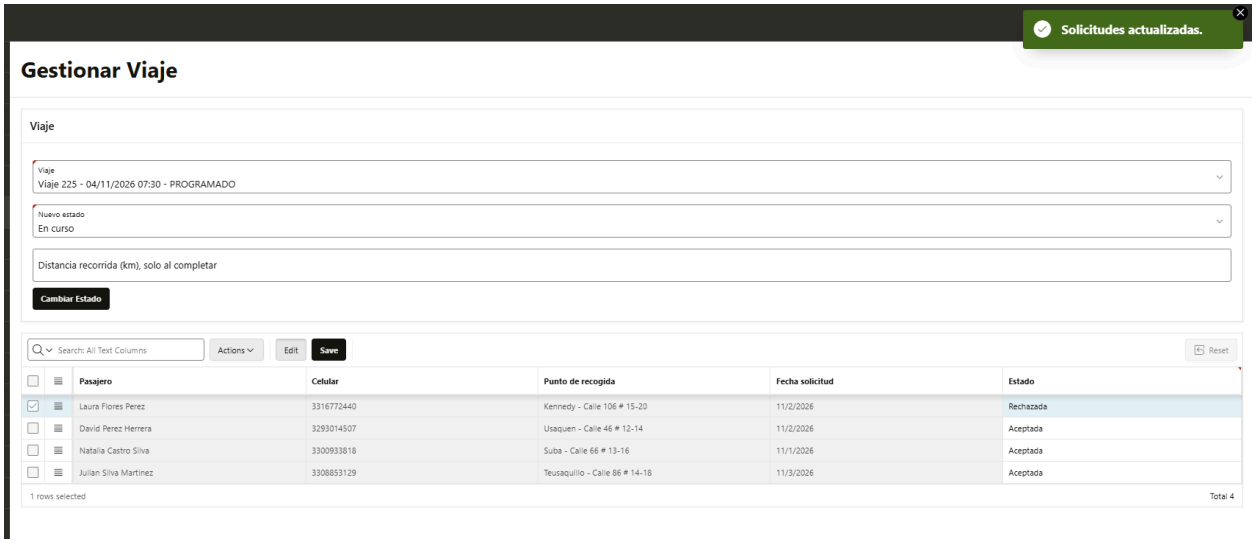
Mis Viajes

Id Viaje	Salida	Placa	Ruta General	Tarifa	Cupos Ofrecidos	Estado	Confirmados	Pasajeros Confirmados
461	19/11/2026 06:45	JJM269	Toda la séptima hasta la universidad	5	4	PROGRAMADO	0	
226	11/11/2026 07:30	BDG011	Occidente - Calle 80 - Avenida Boyaca - Universidad	4500	4	EN_CURSO	4	Andres Diaz Ortiz
226	11/11/2026 07:30	BDG011	Occidente - Calle 80 - Avenida Boyaca - Universidad	4500	4	EN_CURSO	4	Andres Gonzalez Ortiz
226	11/11/2026 07:30	BDG011	Occidente - Calle 80 - Avenida Boyaca - Universidad	4500	4	EN_CURSO	4	Camilo Herrera Torres
226	11/11/2026 07:30	BDG011	Occidente - Calle 80 - Avenida Boyaca - Universidad	4500	4	EN_CURSO	4	Valentina Lopez Vargas
462	10/11/2026 08:15	SIO132	Autopista y NQS	6	2	PROGRAMADO	0	
225	04/11/2026 07:30	BDG011	Norte - Autopista Norte - Calle 100 - Universidad	4000	4	PROGRAMADO	3	David Perez Herrera
225	04/11/2026 07:30	BDG011	Norte - Autopista Norte - Calle 100 - Universidad	4000	4	PROGRAMADO	3	Julian Silva Martinez
225	04/11/2026 07:30	BDG011	Norte - Autopista Norte - Calle 100 - Universidad	4000	4	PROGRAMADO	3	Natalia Castro Silva
441	02/11/2026 18:00	VLC231	Calle 26 hacia Uniandes	8000	3	PROGRAMADO	0	
224	06/10/2026 07:30	BDG011	Suroccidente - Avenida Boyaca - Calle 13 - Universidad	6500	4	CANCELADO	0	
223	15/09/2026 07:30	BDG011	Noroccidente - Avenida Suba - Calle 127 - Universidad	6000	4	COMPLETADO	3	Mariana Herrera Moreno
223	15/09/2026 07:30	BDG011	Noroccidente - Avenida Suba - Calle 127 - Universidad	6000	4	COMPLETADO	3	Santiago Flores Castro
223	15/09/2026 07:30	BDG011	Noroccidente - Avenida Suba - Calle 127 - Universidad	6000	4	COMPLETADO	3	Sebastian Lopez Rojas
222	08/09/2026 07:30	BDG011	Centro - Carrera 7 - Calle 45 - Universidad	5500	4	COMPLETADO	3	Daniel Rodriguez Garcia

1 - 15 Next ▸

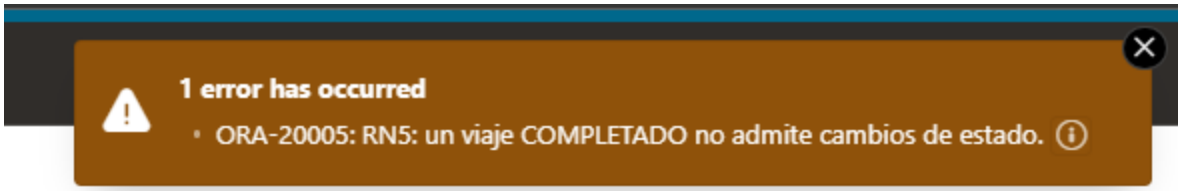
Gestionar viaje

Prueba exitosa

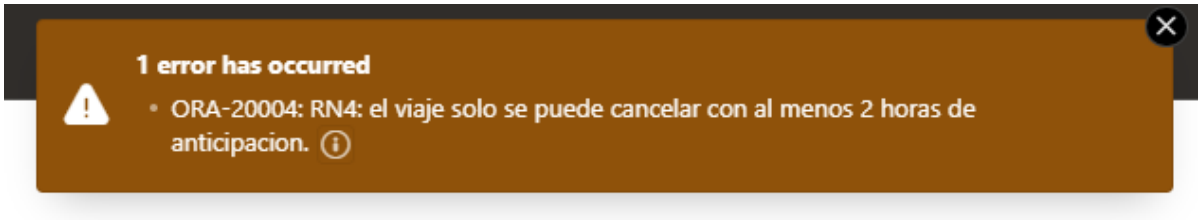


Prueba fallida



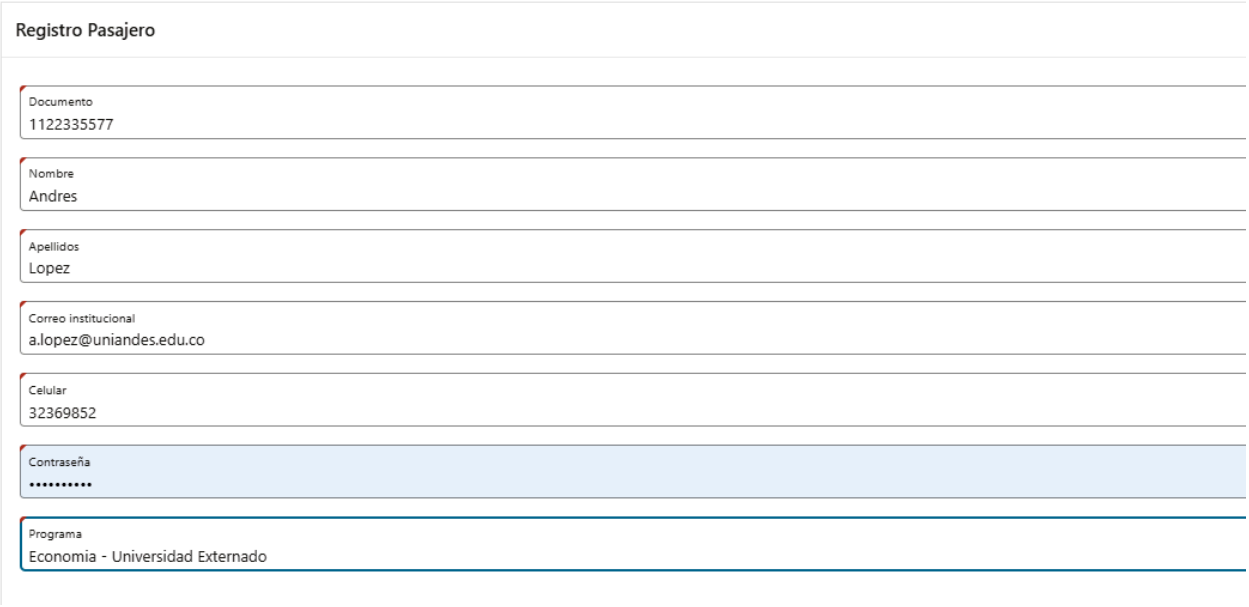


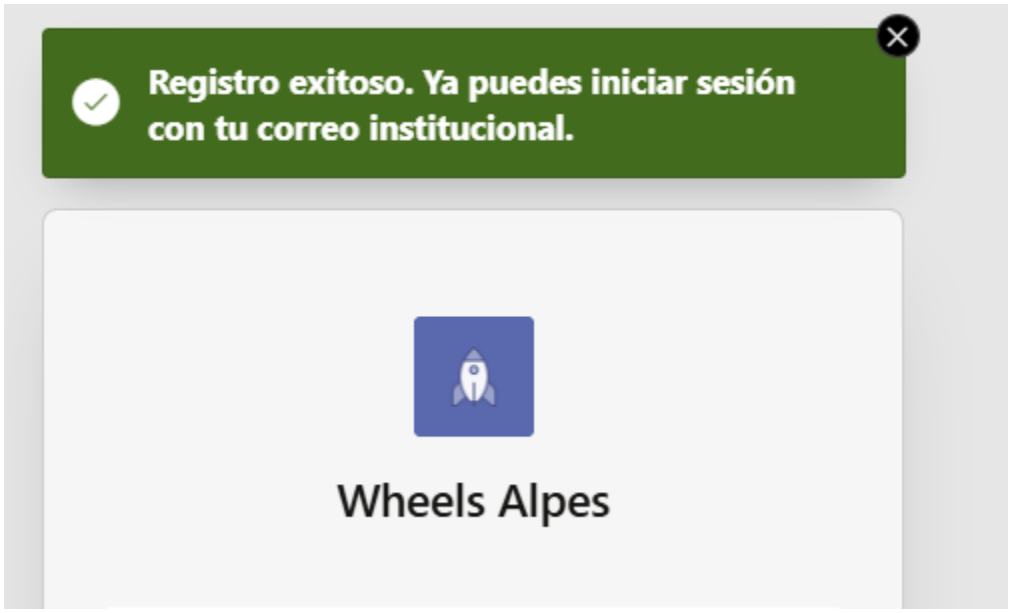




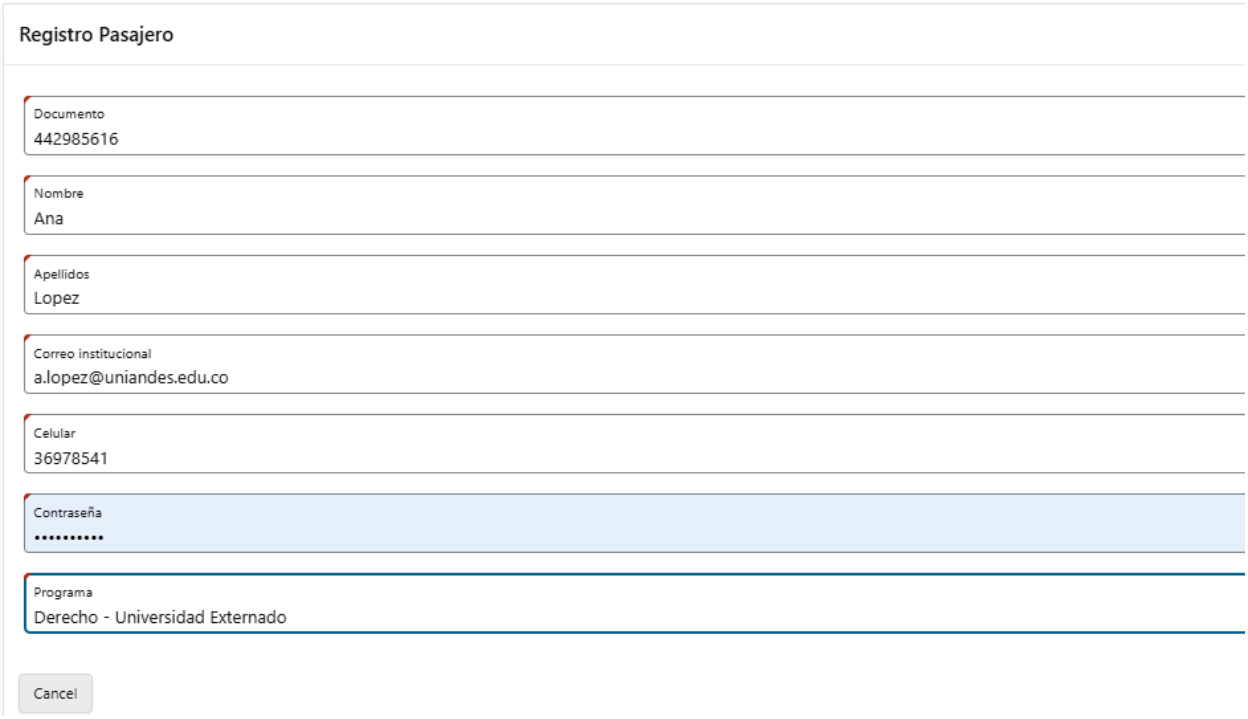
Registro pasajero

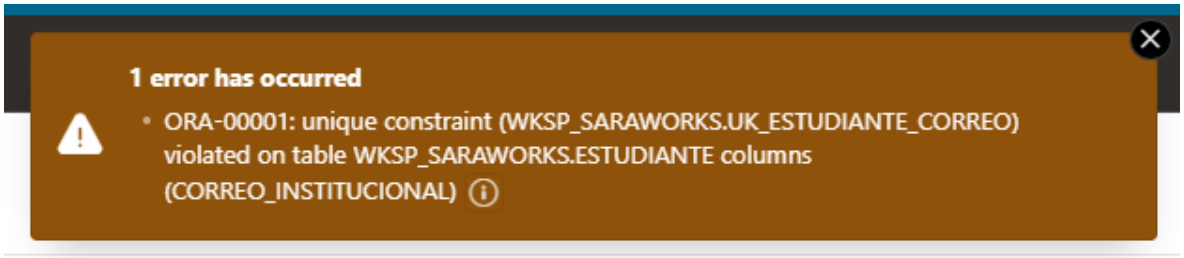
Prueba exitosa





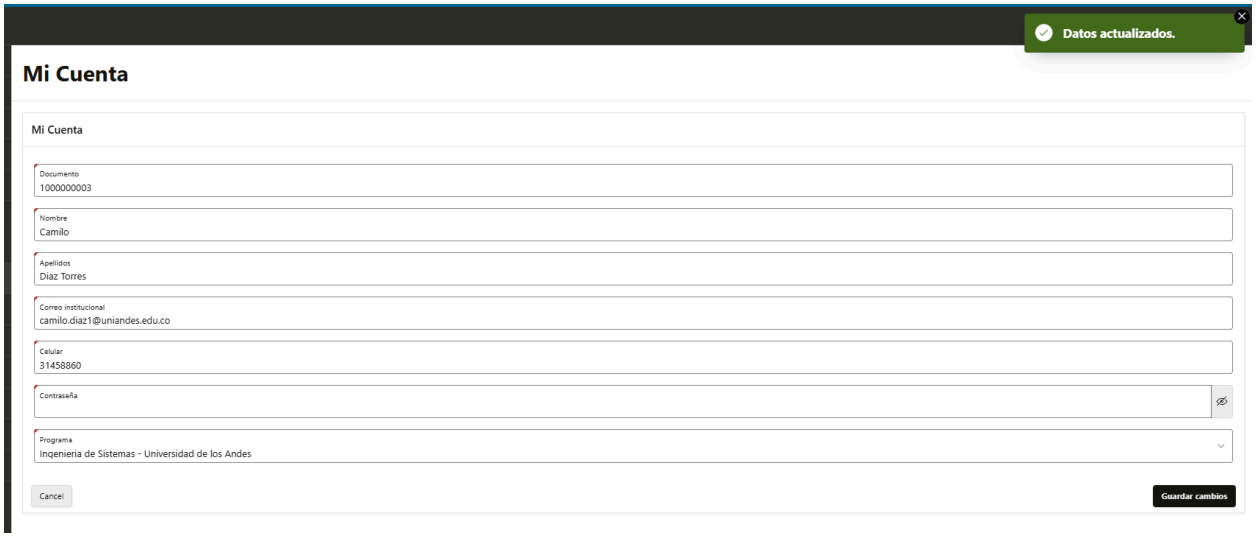
Prueba fallida



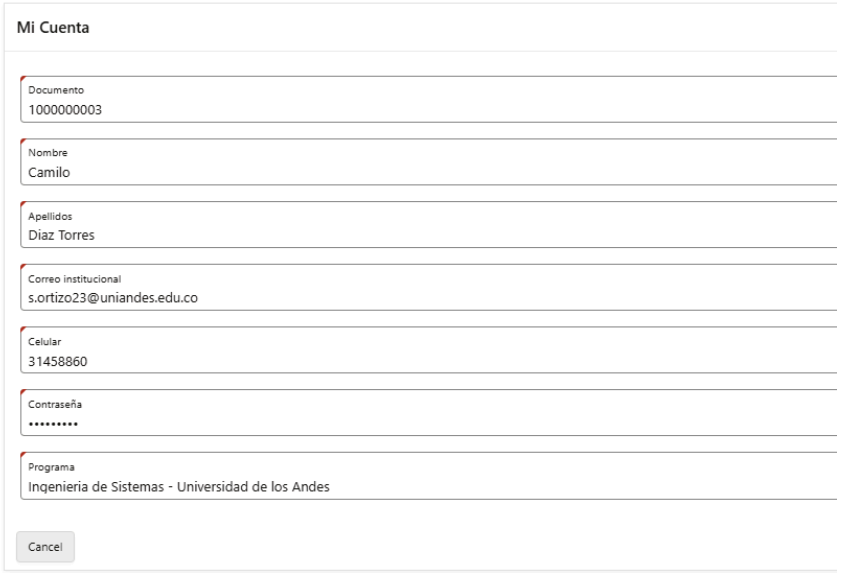


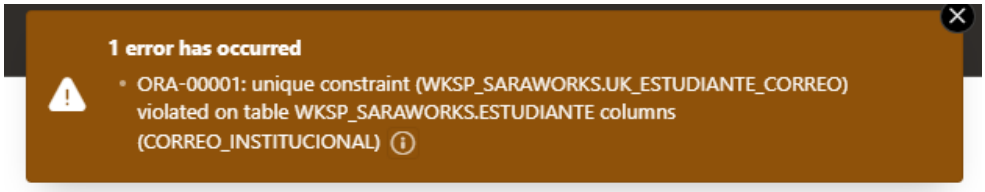
Mi cuenta

Prueba exitosa



Prueba fallida



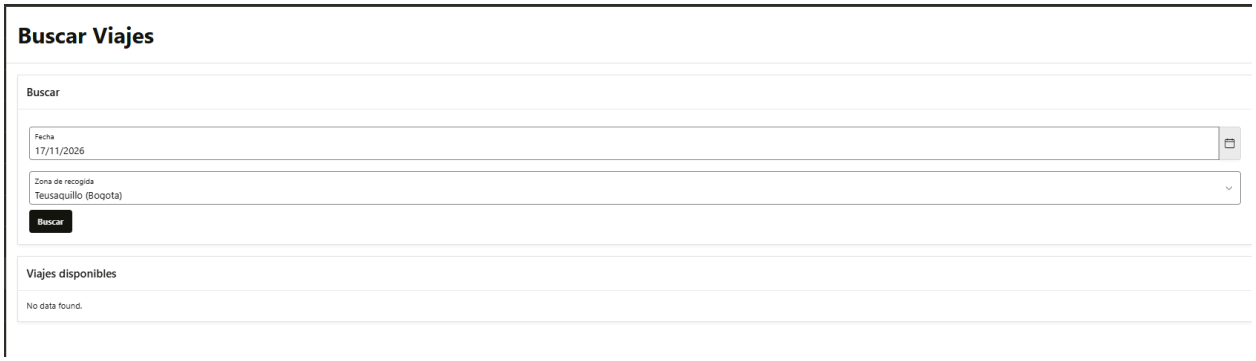


Buscar viajes

Prueba exitosa



Prueba vacía

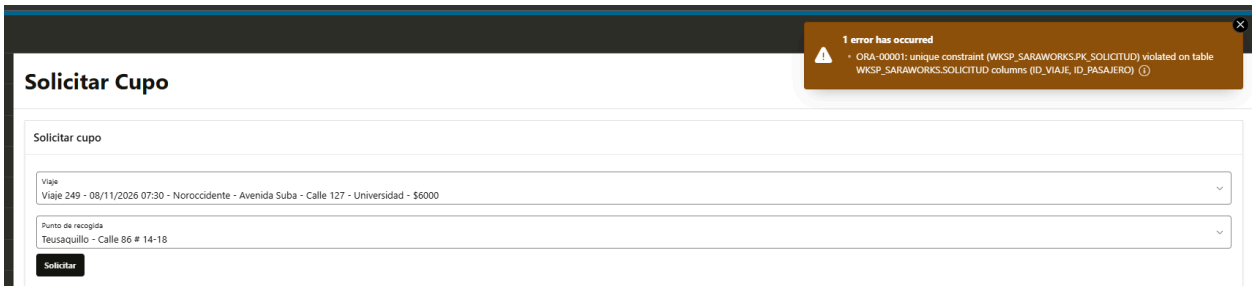


Solicitar cupo

Prueba exitosa

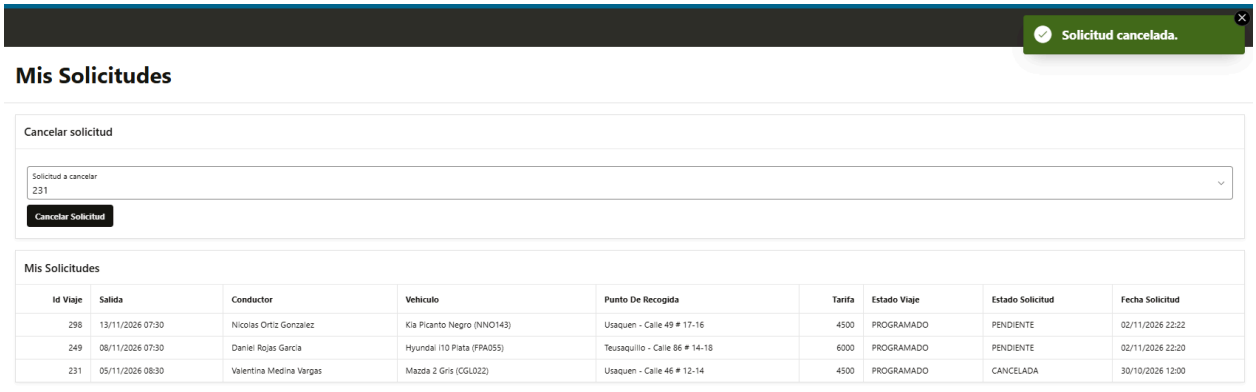


Prueba fallida



Mis solicitudes

Prueba exitosa



Pruebas SQL generales

```
1
2   SELECT * FROM vehiculo ORDER BY placa DESC FETCH FIRST 5 ROWS ONLY;
```

Results | Explain | Describe | Saved SQL | History

PLACA	ID_CONDUCTOR	MARCA	MODELO	ANIO	COLOR	CUPOS
YUR264	224	Toyota	Yaris	2012	Blanco	4
XRM253	223	Kia	Picanto	2023	Plata	6
WOH242	222	Mazda	2	2022	Azul	5
VLC231	221	Chevrolet	Spark	2021	Rojo	4
UIX220	240	Renault	Sandero	2020	Gris	6

```
1
2   SELECT * FROM viaje ORDER BY id_viaje DESC FETCH FIRST 5 ROWS ONLY;
```

Results | Explain | Describe | Saved SQL | History

...JE	PLACA	ID_PUNTO_ORIGEN	FECHA_HORA_SALIDA	RUTA_GENERAL	TARIFA	CUPOS_OFRECIDOS	ESTADO	DISTANCIA_KM	HORA_SALIDA_REAL	HORA_LLEGADA_REAL
	JJM269	224	19-NOV-26 06.45.00.000000 AM	Toda la séptima hasta la universidad	5	4	PROGRAMADO	-	-	-
	VLC231	248	02-NOV-26 06.00.00.000000 PM	Calle 26 hacia Uniandes	8000	3	COMPLETADO	-	02-NOV-26 10.05.04.570215 PM	02-NOV-26 10.06.04.408830 PM
	UIX220	227	10-NOV-26 06.30.00.000000 AM	Sur - Avenida Caracas - Calle 26 - Universidad	5000	3	PROGRAMADO	-	-	-

```
SELECT * FROM parada_viaje WHERE id_viaje = 225 ORDER BY orden;
```

Results | Explain | Describe | Saved SQL | History

ID_VIAJE	ID_PUNTO	ORDEN	ORIGEN_PARADA
225	241	1	INICIAL
225	248	2	INICIAL
225	225	3	SOLICITUD
225	228	4	SOLICITUD
225	231	5	SOLICITUD

5 rows returned in 0.01 seconds Download

```
SELECT id_viaje, id_pasajero, estado FROM solicitud WHERE id_viaje = 225;
```

Results | Explain | Describe | Saved SQL | History

ID_VIAJE	ID_PASAJERO	ESTADO
225	257	ACEPTADA
225	258	ACEPTADA
225	259	ACEPTADA
225	260	RECHAZADA

4 rows returned in 0.00 seconds Download

```
SELECT id_estudiante, documento, correo_institucional, celular FROM estudiante ORDER BY id_estudiante DESC FETCH FIRST 5 ROWS ONLY;
```

Results | Explain | Describe | Saved SQL | History

ID_ESTUDIANTE	DOCUMENTO	CORREO_INSTITUCIONAL	CELULAR
484	12345678	a.lozano1@uniandes.edu.co	321456987
482	442985616	a.lozano@uniandes.edu.co	36978541
462	1122554477	a.lopez@uniandes.edu.co	32369852
461	1234567890	s.ortizo23@uniandes.edu.co	3123456789
441	1099999999	laura.prueba@uniandes.edu.co	3001234567

Prueba de llave foránea

Prueba exitosa

```
1   INSERT INTO vehiculo (placa, id_conductor, marca, modelo, anio, color, cupos)
2   VALUES ('FKT001', (SELECT MIN(id_estudiante) FROM conductor), 'Mazda', '3', 2022, 'Gris', 4);
```

Results | Explain | Describe | Saved SQL | History

```
1 row(s) inserted.
```

```
1   SELECT placa, id_conductor, marca, modelo FROM vehiculo WHERE placa = 'FKT001';
```

Results | Explain | Describe | Saved SQL | History

PLACA	ID_CONDUCTOR	MARCA	MODELO
FKT001	221	Mazda	3

1 rows returned in 0.01 seconds Download

Prueba fallida

```
INSERT INTO vehiculo (placa, id_conductor, marca, modelo, anio, color, cupos)
VALUES ('FKT002', 999999, 'Mazda', '3', 2022, 'Gris', 4);
```

sults | Explain | Describe | Saved SQL | History

```
ORA-02291: integrity constraint (WKSP_SARAWORKS.FK_VEHICULO_CONDUCTOR) violated -
parent key not found
```

```
1   SELECT COUNT(*) FROM vehiculo WHERE placa = 'FKT002';
```

Results | Explain | Describe | Saved SQL | History

COUNT(*)
0

1 rows returned in 0.01 seconds Download

Escenario	Qué se probó	Resultado esperado	Resultado obtenido
Registro conductor, éxito	Registrar un conductor nuevo con su vehículo (RF1)	Se guardan estudiante, conductor y vehículo, y la app confirma	Mensaje “Registro exitoso”; la consulta SQL devuelve una fila con el estudiante 461 y su vehículo JJJ444
Registro conductor, fallo	Unicidad: registrar un conductor con un documento ya existente	Falla con ORA-00001 y no se guarda nada	ORA-00001 sobre UK_ESTUDIANTE_DOCUMENTO
Vehículo, éxito	Guardar un vehículo con datos válidos (RF2)	Se guarda y aparece en la lista	“Changes saved”; el vehículo JJM269 Toyota Prado de 4 cupos aparece en Mis Vehículos
Vehículo, fallo	CHECK: guardar un vehículo con 0 cupos y año 2046	Falla con ORA-02290 y no se guarda	ORA-02290 (CK_VEHICULO_CUPOS); el vehículo queda sin cambios. Oracle reporta sólo la primera restricción violada
Publicar viaje, éxito	Viaje de 4 cupos en un vehículo de 4, con 2 paradas iniciales (RF3)	Se crea el viaje PROGRAMADO con sus paradas	“Viaje publicado correctamente”; aparece en Mis Viajes y en la consulta SQL como PROGRAMADO
Publicar viaje, fallo	RN1: ofrecer 5 cupos en un vehículo de 3	Falla con ORA-20001 y no se crea el viaje	“ORA-20001: RN1: el viaje no puede ofrecer más cupos (5) que los del vehículo (3)”
Mis viajes	Consultar el historial del conductor (RF6)	Lista de sus viajes con estado y pasajeros confirmados	Se listan los viajes propios con su estado y los pasajeros confirmados
Gestionar viaje, éxito	RF4: rechazar una solicitud pendiente del viaje 225	Pasa a RECHAZADA y no se crea una parada nueva	“Solicitudes actualizadas”; SQL: 3 ACEPTADA y 1 RECHAZADA; el viaje sigue con 5 paradas (3 de origen SOLICITUD, RN8)

Gestionar viaje, fallo 1	RN5: cambiar el estado de un viaje COMPLETADO (223)	Falla con ORA-20005	“ORA-20005: RN5: un viaje COMPLETADO no admite cambios de estado”
Gestionar viaje, fallo 2	RN4: cancelar un viaje que sale en menos de 2 horas (441)	Falla con ORA-20004	“ORA-20004: RN4: el viaje solo se puede cancelar con al menos 2 horas de anticipación”
Registro pasajero, éxito	Crear una cuenta de pasajero (RF7)	Se guarda y se puede iniciar sesión	“Registro exitoso”; la consulta muestra a.lopez@uniandes.edu.co (id 462)
Registro pasajero, fallo	Unicidad: registrar otro pasajero con un correo ya usado	Falla con ORA-00001	ORA-00001 sobre UK_ESTUDIANTE_CORREO
Mi cuenta, éxito	Modificar el celular (RF8)	Se actualizan los datos	“Datos actualizados”
Mi cuenta, fallo	Cambiar el correo a uno que ya tiene otro usuario	Falla con ORA-00001	ORA-00001 sobre UK_ESTUDIANTE_CORREO
Buscar viajes, con datos	Buscar por fecha y zona (RF9)	Muestra los viajes programados de esa zona y fecha	Se lista el viaje 231
Buscar viajes, vacía	Buscar una fecha sin viajes	Resultado vacío sin error	“No data found”
Solicitar cupo, éxito	Pedir cupo en el viaje 249 con un punto de recogida (RF10)	Queda una solicitud PENDIENTE	“Solicitud enviada. El conductor debe aceptarla”
Solicitar cupo, fallo	RN9: pedir cupo dos veces en el mismo viaje	Falla con ORA-00001	ORA-00001 sobre PK_SOLICITUD (id_viaje, id_pasajero)

Mis solicitudes, éxito	Cancelar una solicitud con más de 2 horas de anticipación (RF11)	Pasa a CANCELADA	“Solicitud cancelada”; la lista muestra CANCELADA